



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 8

Contador Ascendente con Reinicio
usando Display de 7 Segmentos
en Raspberry Pi Pico W

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

27 de Febrero de 2026

Contents

1	Introducción	2
1.1	Objetivo	2
1.2	Trabajo Previo	2
2	Desarrollo	2
2.1	Diseño del Circuito	2
2.2	Tabla de Segmentos Hexadecimales	3
2.3	Proceso de Desarrollo con PlatformIO	3
2.4	Errores Comunes y Soluciones	4
3	Preguntas de Repaso	4
3.1	Equivalencia binaria de los dígitos hexadecimales en <code>hexDigits[]</code> . . .	4
3.2	Explicación de <code>int32_t mask = hexDigits[num] << FIRST_GPIO</code> . . .	5
3.3	Rol de <code>gpio_set_mask()</code> y ausencia de resistencia en el botón	5
4	Conclusión	5
	Anexos	6

1 Introducción

En esta práctica se amplía el contador binario de la práctica anterior incorporando un **display de 7 segmentos de ánodo común** que muestra el valor actual del contador en representación hexadecimal (0–F), mientras los 4 LEDs rojos continúan indicando el mismo valor en binario. El conteo avanza con cada pulsación del botón y se reinicia automáticamente al superar el valor 15 (F en hexadecimal), ciclo gestionado íntegramente por software en la Raspberry Pi Pico W mediante **PlatformIO** con el framework Arduino.

1.1 Objetivo

Aprender la equivalencia entre valores hexadecimales y binarios, comprender el uso de máscaras de bits para controlar múltiples pines de forma simultánea e identificar la distribución de segmentos de un display de 7 segmentos de ánodo común para representar dígitos mediante patrones de bits.

1.2 Trabajo Previo

Como preparación se analizó la conexión del display de 7 segmentos de ánodo común. En este tipo de display el ánodo de todos los LEDs internos está unido a V_{CC} ; para encender un segmento se lleva su pin de cátodo a nivel bajo (LOW). Los 7 segmentos se etiquetan de la **a** a la **g** según la disposición estándar mostrada en la Figura 1.

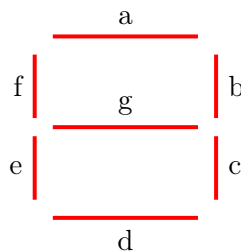


Figure 1: Distribución estándar de segmentos en un display de 7 segmentos

Adicionalmente, se creó la carpeta de proyecto **Practica_8** en PlatformIO, estableciendo la estructura necesaria: `platformio.ini`, `src/main.cpp`, y la selección del framework *Arduino* para la placa Raspberry Pi Pico W.

2 Desarrollo

2.1 Diseño del Circuito

El circuito combina los elementos de la práctica anterior con el display de 7 segmentos:

- **4 LEDs rojos** conectados a GP6 (MSB), GP7, GP8 y GP9 (LSB), cada uno con resistencia limitadora de 200Ω .

- **Display de 7 segmentos de ánodo común** con los segmentos A–G conectados a los pines GP28, GP27, GP21, GP20, GP19, GP26 y GP22 respectivamente, protegido con una resistencia de $220\ \Omega$ en la línea de ánodo común.
- **Pulsador sin enclavamiento** en GP15, configurado con resistencia interna *pull-up*; activo en nivel bajo (LOW). Se añade una resistencia externa de $1\ \text{k}\Omega$ en serie como protección adicional del pin.

El cálculo de la resistencia para los LEDs sigue el mismo criterio de la práctica anterior:

$$R = \frac{V_{CC} - V_{LED}}{I_{LED}} = \frac{3.3 - 2.0}{0.0065} \approx 200\ \Omega$$

Para el display, considerando una caída de $2.0\ \text{V}$ y una corriente de segmento de $5\ \text{mA}$:

$$R_{display} = \frac{3.3 - 2.0}{0.005} = 260\ \Omega \rightarrow 220\ \Omega \text{ (valor comercial disponible)}$$

2.2 Tabla de Segmentos Hexadecimales

Cada dígito hexadecimal se codifica como un byte donde los bits 0 a 6 corresponden a los segmentos a–g respectivamente. Un bit en 1 indica segmento encendido. Para el display de ánodo común la lógica se invierte en software (!bit).

Table 1: Tabla de patrones de segmentos para dígitos hexadecimales

Dígito	Hex	Binario (gfedcba)	a	b	c	d	e	f	g
0	0x3F	0011 1111	1	1	1	1	1	1	0
1	0x06	0000 0110	0	1	1	0	0	0	0
2	0x5B	0101 1011	1	1	0	1	1	0	1
3	0x4F	0100 1111	1	1	1	1	0	0	1
4	0x66	0110 0110	0	1	1	0	0	1	1
5	0x6D	0110 1101	1	0	1	1	0	1	1
6	0x7D	0111 1101	1	0	1	1	1	1	1
7	0x07	0000 0111	1	1	1	0	0	0	0
8	0x7F	0111 1111	1	1	1	1	1	1	1
9	0x6F	0110 1111	1	1	1	1	0	1	1
A	0x77	0111 0111	1	1	1	0	1	1	1
b	0x7C	0111 1100	0	0	1	1	1	1	1
C	0x39	0011 1001	1	0	0	1	1	1	0
d	0x5E	0101 1110	0	1	1	1	1	0	1
E	0x79	0111 1001	1	0	0	1	1	1	1
F	0x71	0111 0001	1	0	0	0	1	1	1

2.3 Proceso de Desarrollo con PlatformIO

Al igual que en la práctica anterior, se utilizó PlatformIO como entorno de desarrollo. Los pasos seguidos fueron:

1. **Creación del proyecto:** Desde la interfaz de PlatformIO se seleccionó la placa *Raspberry Pi Pico W* con el framework *Arduino*, generando automáticamente la estructura del proyecto con su `platformio.ini`.
2. **Escritura del código:** Se desarrolló `main.cpp` combinando la lógica de LEDs binarios de la práctica 7 con la nueva función de actualización del display de 7 segmentos.
3. **Compilación y carga:** Con el botón *Build* se compiló el proyecto; posteriormente, con la Pico W en modo BOOTSEL, se utilizó *Upload* para transferir el binario sin manipular manualmente el archivo `.uf2`.
4. **Verificación:** Tras la carga, cada pulsación del botón incrementa el contador mostrando simultáneamente el valor binario en los LEDs y el dígito hexadecimal en el display, reiniciándose al superar 15.

2.4 Errores Comunes y Soluciones

Table 2: Errores encontrados y soluciones aplicadas durante el desarrollo

Error Encontrado	Solución Aplicada
El display permanece apagado o muestra segmentos incorrectos.	Se verificó que el display fuera de ánodo común y que la lógica de la función <code>actualizarDisplay()</code> invirtiera el bit con <code>!bit</code> antes de escribir en el pin (HIGH apaga, LOW enciende).
Algunos segmentos siempre encendidos.	Se inicializaron todos los pines del display en HIGH durante <code>setup()</code> para asegurar el estado apagado inicial.
LEDs y display no sincronizan correctamente.	Se confirmó que ambas funciones (<code>actualizarLEDs</code> y <code>actualizarDisplay</code>) reciben el mismo valor de <code>counter</code> en la misma pulsación detectada.

3 Preguntas de Repaso

3.1 Equivalencia binaria de los dígitos hexadecimales en `hexDigits[]`

La tabla anterior muestra la equivalencia binaria completa de los 16 valores almacenados en `hexDigits[]`. Cada bit del byte corresponde a un segmento físico del display: el bit 0 controla el segmento **a** (superior), el bit 1 el segmento **b** (superior derecho), y así sucesivamente hasta el bit 6 que controla el segmento **g** (central).

La relación directa es que los bits en 1 dentro de cada valor hexadecimal determinan exactamente qué segmentos deben encenderse para formar la figura del número o letra que se desea representar visualmente. Por ejemplo, $0x3F$ ($= 0011\ 1111_2$) enciende los seis segmentos exteriores (a, b, c, d, e, f) y apaga el central (g), formando el dígito **0**.

3.2 Explicación de `int32_t mask = hexDigits[num] « FIRST_GPIO`

Esta línea construye una **máscara de bits** para controlar simultáneamente los pines del display con una sola operación. El operador de desplazamiento `«` mueve todos los bits del patrón de segmentos hacia la izquierda tantas posiciones como indica `FIRST_GPIO` (valor 2 en la práctica base). Esto alinea el bit 0 del patrón con el GPIO 2, el bit 1 con el GPIO 3, etcétera, de modo que la variable `mask` es un entero de 32 bits donde cada bit en posición n indica si el GPIO n debe activarse. Posteriormente, `gpio_set_mask(mask)` escribe todos esos pines en un solo ciclo de instrucción, lo cual es más eficiente que llamar a `gpio_put()` para cada pin de forma individual.

3.3 Rol de `gpio_set_mask()` y ausencia de resistencia en el botón

La función `gpio_set_mask()` recibe un entero de 32 bits y activa en HIGH todos los pines GPIO cuyo bit correspondiente sea 1, ignorando los bits en 0. Su ventaja es la atomicidad: todos los pines se actualizan en el mismo ciclo, evitando destellos o estados intermedios indeseados cuando se cambia el patrón del display.

Respecto al botón, no se coloca una resistencia externa de *pull-up* porque se utiliza la **resistencia interna** del microcontrolador, activada mediante `gpio_pull_up(BUTTON_GPIO)` (SDK) o `pinMode(BTN_PIN, INPUT_PULLUP)` (Arduino). Esta resistencia integrada mantiene el pin en nivel alto cuando el botón está abierto, cumpliendo la misma función que una resistencia externa de 10–50 k Ω , por lo que añadir una segunda resistencia externa sería redundante. No obstante, en el circuito combinado con PlatformIO sí se mantiene la resistencia de 1 k Ω en serie con el pin como protección ante cortocircuitos accidentales.

4 Conclusión

La práctica amplió competencias fundamentales en el desarrollo con sistemas embebidos: uso de un display de 7 segmentos de ánodo común, codificación de dígitos mediante tablas de patrones de bits, y aplicación de máscaras para el control eficiente de múltiples GPIOs. La integración con el contador binario de LEDs permitió comparar dos formas distintas de representar el mismo valor numérico: binaria y hexadecimal, reforzando la comprensión de los sistemas de numeración empleados en electrónica digital.

PlatformIO continuó demostrando su utilidad al simplificar la compilación y la carga del firmware, eliminando la necesidad de gestionar manualmente archivos CMake.

Posibles mejoras:

- **Display multiplexado:** Implementar varios dígitos compartiendo los pines de segmentos mediante multiplexado por tiempo.

- **Contador bidireccional:** Agregar un segundo botón para decrementar, permitiendo contar de F a 0.
- **Detección por interrupción:** Reemplazar el *polling* por una *IRQ* en flanco de bajada para reducir el uso del procesador.
- **Visualización decimal:** Adaptar la tabla de segmentos para mostrar los valores del 0 al 9 en decimal en lugar de hexadecimal.

Anexos

Anexo A – Código Fuente Completo (main.cpp)

```
1  #include <Arduino.h>
2
3  //          LEDs binarios (4 bits)
4
5  #define LED_MSB    6
6  #define LED_B2     7
7  #define LED_B1     8
8  #define LED_LSB    9
9
10 //          Display 7 segmentos (anodo comun)
11
12 #define SEG_A      28
13 #define SEG_B      27
14 #define SEG_C      21
15 #define SEG_D      20
16 #define SEG_E      19
17 #define SEG_F      26
18 #define SEG_G      22
19
20 //          Boton
21
22 #define BTN_PIN      15
23 #define DEBOUNCE_MS  50
24
25 //          Segmentos por digito hex (bit0=A...bit6=G, 1=ON)
26
27 const uint8_t SEG_TABLE[16] = {
28     0x3F, // 0
29     0x06, // 1
30     0x5B, // 2
31     0x4F, // 3
32     0x66, // 4
```

```
29     0x6D,    // 5
30     0x7D,    // 6
31     0x07,    // 7
32     0x7F,    // 8
33     0x6F,    // 9
34     0x77,    // A
35     0x7C,    // b
36     0x39,    // C
37     0x5E,    // d
38     0x79,    // E
39     0x71     // F
40 };
41
42 const uint8_t SEG_PINS[7] = {SEG_A, SEG_B, SEG_C, SEG_D, SEG_E
    , SEG_F, SEG_G};
43 const uint8_t LED_PINS[4] = {LED_MSB, LED_B2, LED_B1, LED_LSB
    };
44
45 uint8_t counter = 0;
46 bool last_btn_state = HIGH;
47
48 //      Actualiza los 4 LEDs binarios
49
49 void actualizarLEDs(uint8_t valor) {
50     for (int i = 0; i < 4; i++) {
51         uint8_t bit = (valor >> (3 - i)) & 0x01;
52         digitalWrite(LED_PINS[i], bit);
53     }
54 }
55
56 //      Actualiza display 7 segmentos (anodo comun = LOW
    activo)
57 void actualizarDisplay(uint8_t valor) {
58     uint8_t patron = SEG_TABLE[valor & 0x0F];
59     for (int i = 0; i < 7; i++) {
60         digitalWrite(SEG_PINS[i], !((patron >> i) & 0x01));
61     }
62 }
63
64 void setup() {
65     for (int i = 0; i < 4; i++) {
66         pinMode(LED_PINS[i], OUTPUT);
67         digitalWrite(LED_PINS[i], LOW);
68     }
69     for (int i = 0; i < 7; i++) {
70         pinMode(SEG_PINS[i], OUTPUT);
```



```
71     digitalWrite(SEG_PINS[i], HIGH); // todos apagados (
        anodo comun)
72 }
73
74 pinMode(BTN_PIN, INPUT_PULLUP);
75
76 // Mostrar estado inicial (0)
77 actualizarLEDs(counter);
78 actualizarDisplay(counter);
79 }
80
81 void loop() {
82     bool btn_state = digitalRead(BTN_PIN);
83
84     if (btn_state == LOW && last_btn_state == HIGH) {
85         delay(DEBOUNCE_MS);
86         if (digitalRead(BTN_PIN) == LOW) {
87             counter = (counter + 1) & 0x0F; // 0-15 y vuelve
                a 0
88             actualizarLEDs(counter); // LEDs
                binarios
89             actualizarDisplay(counter); // Display hex
90
91             while (digitalRead(BTN_PIN) == LOW) {
92                 delay(10);
93             }
94             delay(DEBOUNCE_MS);
95         }
96     }
97
98     last_btn_state = digitalRead(BTN_PIN);
99     delay(10);
100 }
```

Anexo B – Archivo platformio.ini

```
1 [env:rpipicow]
2 platform = raspberrypi
3 board    = rpicow
4 framework = arduino
5
6 ; Velocidad del monitor serial
7 monitor_speed = 115200
8
9 ; Opciones de compilacion
10 build_flags =
```

Anexo C – Diagrama del Circuito en Protoboard

El diagrama siguiente representa esquemáticamente las conexiones del circuito. Los segmentos del display de ánodo común se conectan a través de resistencias de $220\ \Omega$. Los LEDs rojos utilizan resistencias de $200\ \Omega$. El pulsador se conecta con una resistencia de $1\ \text{k}\Omega$ en serie con el pin GP15 (pull-up interno activo).

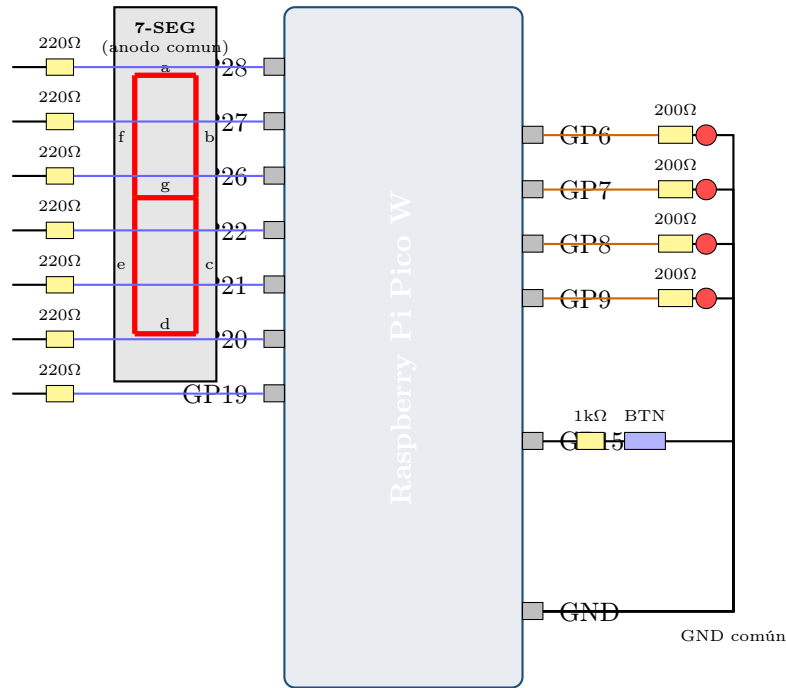


Figure 2: Diagrama esquemático del circuito (representación simplificada). Los segmentos del display de ánodo común se conectan a través de resistencias de $220\ \Omega$; los LEDs rojos con $200\ \Omega$; el pulsador con $1\ \text{k}\Omega$ de protección y pull-up interno activado por software.

Anexo D – Fotografía del Prototipo Armado

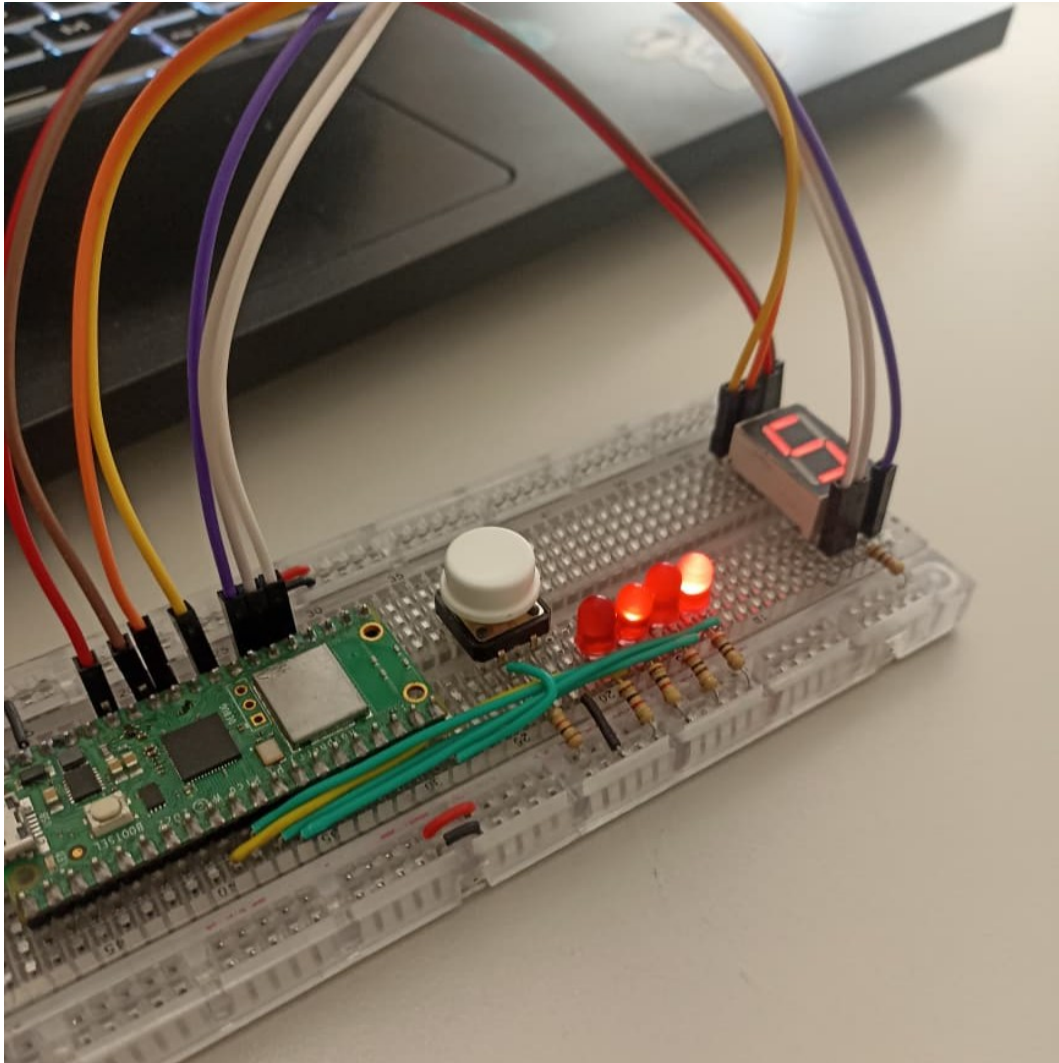


Figure 3: Fotografía del circuito armado en protoboard: Raspberry Pi Pico W con display de 7 segmentos, 4 LEDs rojos y pulsador.

References

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.0). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.

- [4] PlatformIO Labs, *PlatformIO Documentation*. <https://docs.platformio.org>, 2024.
- [5] Texas Instruments, *Seven-Segment Display Fundamentals*. <https://www.ti.com/lit/an/slyt079/slyt079.pdf>, 2012.